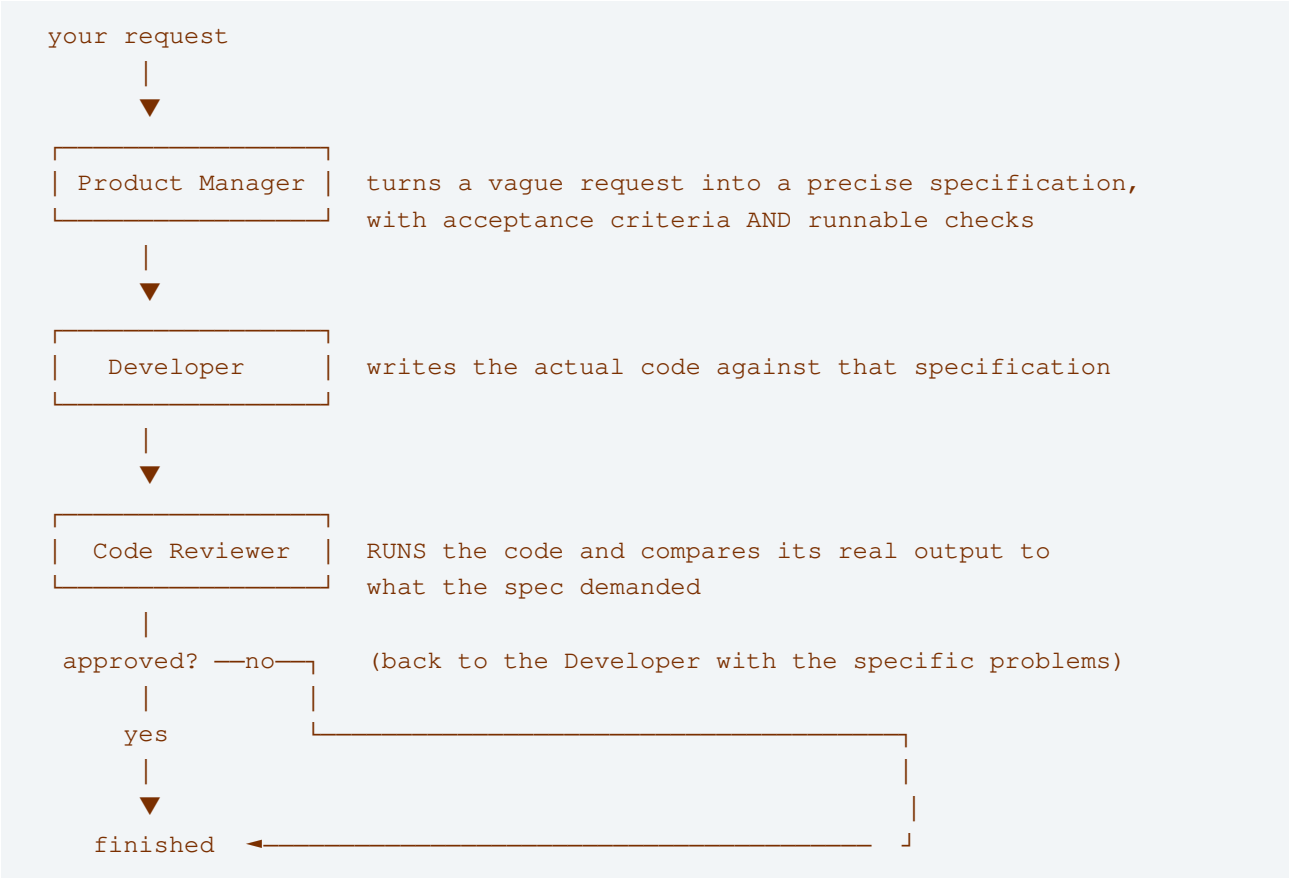
MAWE — how it works, and how to run it

Written so you can explain this project without reading from a script, and run it from a fresh machine without guessing.

Part 1 — The idea

A normal AI coding assistant is **one model answering one prompt**. You ask for a snake game, and in a single pass that model has to understand what you meant, design a solution, write the code, and check its own work. It usually gets something plausible. It also has nobody to catch what it missed, and no reason to look twice.

MAWE splits that into **three specialists and a loop**, the way a real team works:



The single most important design decision is in the Reviewer. **It does not read the code and give an opinion. It runs the code.** The Product Manager emits checks like "run this with the argument `100 C` and standard output must be exactly `212.0`", and the Reviewer executes them in a sandbox. Where the model's opinion and the program's actual behaviour disagree, **the measurement wins.** A model cannot talk its way past a failing program.

That is what makes this different from "ask the AI twice".

Part 2 — The two problems that make it hard

Anyone can wire three prompts together. The reason this is a capstone and not a weekend project is that doing so creates two problems that will destroy you if you ignore them.

Problem 1: the loop might never stop

Two agents disagreeing is an infinite loop. The Developer changes something, the Reviewer finds something else, forever — and every cycle costs money. An unattended run could burn an entire API budget overnight.

So the loop has **five ways to stop**, and a run always exits through exactly one of them:

Exit	Fires when
`approved`	the review passed
`iteration_cap`	the configured maximum was reached (default 5)
`no_progress`	the Developer returned byte-identical code twice — it isn't responding to feedback
`oscillation`	the code went back to a version already seen (A → B → A)
`repeated_issues`	the Reviewer keeps refiling the same complaint
`error`	an agent failed irrecoverably

`oscillation` is the subtle one. If you only check "is this the same as last time?", a loop that alternates between two versions passes that check forever.

How we know it works: 100,000 randomised adversarial review sequences. None failed to terminate, none exceeded its cap.

Problem 2: the conversation grows forever

Every iteration adds the previous code and the previous review to the next prompt. By iteration 20 you are re-sending the entire history every time, and you pay for every token of it.

So older iterations collapse into a structured progress log — what changed, what the review said, which checks passed — while the most recent version is kept in full.

Crucially, this is not done by asking a model to summarise. A summariser that hallucinates would corrupt the Developer's only memory of what it already tried. The fields matter, so they are recorded exactly.

How we know it works: at 80 iterations the prompt falls from 52,494 tokens to 3,316 — a 93.7% reduction — and the outcome does not change.

Part 3 — Setting it up, from nothing

What you need

- **Python 3.11 or newer** — check with `python --version`
- **Git**
- *Optional:* **Node.js**, only for the 8 JavaScript benchmark tasks. Without it they are skipped and everything else still works.
- **No API key.** Nothing in this guide costs money.

Where to run it

Open a terminal — on Windows, PowerShell or the terminal inside VS Code — and go to the project folder. On this machine that is:

```
cd C:\Users\marco\Documents\MAWE-Capstone
```

If you are on a different machine, clone it first:

```
git clone https://github.com/marcoaperozo/Capstone-MAWE.git
```

```
cd Capstone-MAWE
```

Install, once

```
pip install -r requirements.txt
```

```
pip install -e .
```

The second line installs the project itself, so `python -m mawe.cli` works from anywhere in the folder. You only do this once per machine.

Check it worked

```
pytest -q
```

It should finish in about a minute with a line saying everything passed, and some tests **skipped**. Skips here are expected and correct: they are the statistics tests that check our maths against SciPy, which is not a required dependency. Install SciPy and they run too. Nothing is skipped because it is broken.

Deliberately not printed here: the exact number of tests. It changes as the project grows, and a number written into a guide goes stale — this project had to correct exactly that mistake four times. The figure that *is* kept current lives in `docs/reports/FINAL-REPORT.md`, which is generated from measurements.

Part 4 — Running it

Watch the agents work

```
python -m mawe.cli "Build a terminal snake game" --verbose
```

You will see this:

```
  [plan]      specification written, 5 acceptance criteria
              1. the program runs without raising an exception
              2. the snake grows by exactly one segment per food eaten
              3. colliding with a wall ends the game
              ...
  [develop]   iteration 1: src/main.py
  [review]    FAIL  severity=major  tests 2 passed / 0 failed
              1. criterion 4 fails: self-collision never ends the game
  [develop]   iteration 2: src/main.py
  [review]    FAIL  severity=minor  tests 2 passed / 0 failed
              1. all 5 criteria pass; the name 'grow' shadows a local
  [finalize]  approved
```

Read that carefully, because it is the whole project in one screen. The Reviewer rejected the first attempt for a specific, concrete reason. The Developer fixed that exact thing. The second review found only a style nit — severity `minor`, which does not block — so the run was approved.

Useful options

Option	What it does
`--verbose`	show each step as it happens
`--out ./generated`	write the finished code to a folder so you can run it
`--db sqlite:///demo.db`	use a specific database file
`--no-sandbox`	skip executing the code (faster, less evidence)
`--max-iterations 3`	change the loop cap

To actually keep and run the generated program:

```
python -m mawe.cli "Build a command-line calculator" --out ./generated
```

Part 5 — Showing progress

Four independent records, and they agree with each other.

1. The git history — the week-by-week story

```
git log --oneline
```

One commit per week, each carrying that week's result in its message. This is the clearest evidence that the project was built steadily rather than the night before.

2. The database — every run is traceable

Every run records what happened, in 11 tables. After running the CLI with `--db sqlite:///demo.db`:

```
python -c "import sqlite3; c=sqlite3.connect('demo.db'); [print(r) for r in c.execute(
```

```
(0, 'product_manager', '{\n  "summary": "Terminal snake game on a 20x20 grid.",\n  "la
(1, 'developer', 'produced src/main.py')
(1, 'reviewer', 'criterion 4 fails: self-collision never ends the game')
(2, 'developer', 'produced src/main.py')
(2, 'reviewer', "all 5 criteria pass; the name 'grow' shadows a local")
```

Read the first column. The Product Manager spoke once, before any iteration. Then Developer and Reviewer alternate, and **each review is filed against the iteration it actually judged** — which is worth pointing out, because it was wrong until recently: a stale database relationship meant every review after the first was recorded against iteration 1. The run finished correctly, so only the record was wrong, and the record is what we offer as proof a run is traceable.

Every version of the code is kept with a SHA-256 hash — that hash is what the no-progress detector compares.

3. Continuous integration — it works on machines that are not yours

Every push runs the full suite on Python 3.11, 3.12 and 3.13 on GitHub's servers. The green ticks at the Actions tab are independent proof: nothing is passing because of something peculiar to your laptop.

4. The measured results

Command	What it proves	Time
`pytest -q`	everything works	~1 min
`python scripts/how_we_test.py`	how it is tested with no API key	20 s
`python scripts/week6_loop_stress.py`	the loop always terminates	~2 s
`python scripts/week7_memory_check.py`	the context stays bounded	~5 s
`python scripts/week9_bench_check.py`	the benchmark is valid	~2 min
`python scripts/week11_experiment.py`	the four-condition comparison	~3 min
`python scripts/week12_final_check.py`	the final gate, all of it	~2 min

That last one rewrites `docs/reports/findings.json`, which the final report and the deck are

generated from. **No figure in either document is typed by hand** — rerun it and both can be rebuilt.

Part 6 — The honest boundary

Everything above runs against a **mock provider**: a stand-in that replays scripted answers instead of calling a real AI model.

That is a deliberate choice, not a shortcut, and the distinction to hold on to is this: **testing the machinery around a model and testing the quality of a model are two different jobs.** Routing, loop prevention, sandboxed execution, scoring, persistence and token accounting are all machinery, and the mock tests them perfectly. Some of them can *only* be tested this way — proving the no-progress exit fires requires something to return byte-identical code twice, and no real model will do that reliably.

What it does **not** show is whether a real model writes good code. That needs an API key, and it has not been run. The benchmark for it is built and validated — 20 tasks, 181 hidden tests, four conditions, 240 runs — so what is missing is the key, not the code.

Run `python scripts/how_we_test.py` to see that argument demonstrated rather than asserted.

If something goes wrong

Problem	Fix
`python` not recognised	Python isn't installed or isn't on PATH. Reinstall and tick "Add Python to PATH"
`No module named mawe`	You skipped `pip install -e .`, or you are in the wrong folder
`pytest` not recognised	`pip install -r requirements.txt`
JavaScript tasks skipped	Node isn't installed. Harmless — the 12 Python tasks still run
A script seems stuck	The benchmark runs real programs; week 9 takes ~2 minutes. Add `--quick`
27 tests skipped	Correct. Those compare our statistics against SciPy, which is optional